

Rapport d'avancement · 30 septembre 2026

Audit et corrections

PER MEA S9P-2026, drone voilier autonome

Auteur : Haitam Kilani

1 Contexte

Ce rapport couvre la période du 24 au 30 septembre 2026. Je n'ai fait aucun commit du 24 au 29 : tout le travail décrit ici date du 30 septembre et n'est pas encore committé (34 fichiers modifiés, environ 1 460 lignes ajoutées, plus 5 nouveaux fichiers). Je suis parti de l'état du 23 septembre, où tous les tests passaient, ce qui ne m'a pas paru suffisant pour dire que le projet était fiable : j'ai donc relu le firmware, la simulation, l'IHM et les tests en cherchant ce que les tests ne vérifiaient pas, et j'ai reproduit chaque défaut avant de le corriger.

2 Firmware

J'ai corrigé plusieurs défauts dans `main/src/`. Dans `navigation.h`, la normalisation d'angle se faisait avec une boucle `while` dont la durée dépend de la valeur d'entrée : avec une valeur extrême, j'ai mesuré près de 6 millions d'itérations (14,6 ms sur mon PC, donc probablement plusieurs secondes sur l'ESP32, ce qui bloquerait la boucle de commande). Je l'ai remplacée par un calcul en `fmod` en temps constant, et j'ai corrigé `nav_relativeAngle` et `nav_oppositeAngle`, qui ne corrigeaient l'angle qu'une seule fois et donnaient par exemple 9639° pour un vent à 9999°.

Dans `LoRaComm.cpp`, les données reçues par radio n'étaient pas contrôlées. Le vent passait par `atoi` sans vérification de plage. Pour les waypoints, `atof` renvoie 0 sur une trame tronquée, ce qui créait un waypoint valide au large de l'Afrique. J'ai ajouté la validation du vent (valeur numérique, plage, normalisation dans $[0, 360)$) et celle des waypoints (bornes de latitude et de longitude, rejet du point (0,0), et mission acceptée en entier ou refusée en entier plutôt que tronquée).

Dans `DroneApp.cpp`, l'état de navigation (ancres du couloir, phase d'anti-empannage, watchdog) n'était pas réinitialisé quand on quittait le mode Automatique pour le mode Manuel. Il l'est maintenant à cette transition, mais pas entre Failsafe et Automatique, qui suivent tous les deux la mission. Le trim appris du safran est conservé volontairement, car c'est une propriété du matériel. Enfin, dans `AutoController.cpp`, le plafond ESC automatique était `ESC_MAX_US` (2000 μ s) au lieu de `AUTO_ESC_MAX_US` (1850 μ s), et j'ai supprimé quelques avertissements de compilation (`MissionManager.cpp`).

3 Le point qui reste ouvert : la course du safran

En relisant le code, j'ai vu que la constante `ROTOR_AUTO_RANGE_DEG = 20°`, que j'avais validée au banc le 4 juin et flashée sur le bateau, n'existe plus. Le mode automatique autorise maintenant $\pm 110^\circ$ (`ROTOR_AUTO_MIN_US = 1399`, `ROTOR_AUTO_MAX_US = 1601`), soit plus que les $\pm 90^\circ$ que j'avais trouvés trop agressifs en juin et plus que la course de $\pm 90^\circ$ notée dans `Calibration.h`. J'ai lancé un balayage de 5184 géométries sur la vraie machine à états : avec un trim à zéro la commande ne dépasse jamais 90° (pire cas $80,1^\circ$), mais avec un trim chargé à $\pm 40^\circ$ elle dépasse 90° dans 10 à 13 % des cas et sature à 110° dans jusqu'à 10 % des cas.

Je n'ai touché à aucune constante d'actionneur, parce que la butée réelle du winch ne se décide pas dans le code : il faut la mesurer au banc. C'est le prérequis le plus important avant un essai autonome.

4 Simulation

J'ai corrigé plusieurs problèmes du simulateur. Les six scénarios partageaient un même état de navigation statique (`nav_handleNavigation`) et n'étaient donc pas indépendants : chaque `SimulatedBoat` a maintenant son propre `NavState`, remis à zéro au départ. L'argument de scénario était ignoré (`./boat_simulator 3` lançait les six), il est maintenant pris en compte. Le Makefile ne surveillait pas `sim_html_export.hpp`, ce qui laissait des objets périmés, et le Makefile de `IHM/AutoBoat_Simulation` pointait vers un dossier inexistant.

Le plus important : le simulateur ne modélisait ni courant ni dérive, et confondait route sur le fond et cap. Or le firmware pilote sur la route GPS, faute de compas, donc « 6 scénarios sur 6 convergent » ne disait rien du risque principal, le courant de marée. J'ai ajouté la vitesse et la route sur le fond (`courseOverGround`, `speedOverGround`), un courant uniforme (`./boat_simulator N vitesse cap`) et une dérive sous le vent (`SIM_LEEWAY_GAIN`, valeur de référence 0,040, illustrative et non mesurée). Sans option, rien ne change : le scénario 6 donne toujours 688 s. J'ai aussi fait renvoyer un code de sortie non nul en cas d'échec et un tableau PASS/FAIL, pour que le simulateur serve de test de non-régression.

J'ai enfin ajouté un vent imposé (`SIM_WIND_DIR`, `SIM_WIND_SPEED`) et une cible `make run-windy` qui récupère la prévision réelle de Windy avec `simulation/fetch_wind.py`. Cette dernière ne marche pas encore : Windy répond « Invalid API key » avec la clé actuelle de `IHM/.env`, et il faut sans doute une clé de type Point Forecast. J'ai vérifié la conversion des composantes u/v en direction et vitesse avec une fausse réponse.

5 Harnais adverse

J'ai corrigé un plantage (SIGSEGV) quand le fichier CSV de sortie n'est pas inscriptible. J'ai aussi refait tourner le harnais avec 200 missions par valeur et j'ai constaté que les chiffres de `docs/ADVERSARIAL_FINDINGS.md` ne se reproduisaient pas, avec 9 à 13 points d'écart sur du code déterministe. Le générateur aléatoire était en outre réamorcé pour chaque valeur balayée, si bien que chaque point tirait des missions différentes ; il dépend maintenant seulement du nom du balayage, donc tous les points rejouent les mêmes missions. J'ai régénéré le document à partir d'une exécution réelle (deux passes identiques octet pour octet) en précisant N et les intervalles de confiance.

Après recalcul, deux conclusions sont solides : le courant de marée (73,5 % de convergence sans courant, 25,5 % à 0,25 m/s) et l'erreur sur le vent (75,5 % sans erreur, 49,0 % à 20°). La vitesse du winch est sans effet mesurable. En revanche, on ne peut rien conclure sur le bruit GPS avec l'ancien balayage.

6 IHM

Côté serveur, j'ai ajouté le contrôle des coordonnées (route refusée en entier au premier point invalide, même règle dans `IHM_desktop`), des index MongoDB sur `status` et sur `origin + timestamp`, et un envoi des commandes dans l'ordre d'arrivée : sans tri, un Stop et un Navigate en attente en même temps pouvaient partir dans le mauvais ordre. `serial_link.py` (IHM web et

desktop) perdait les trames coupées entre deux lectures ; le résidu est maintenant mis en mémoire jusqu'à la fin de ligne, avec une borne de 4 Ko et une remise à zéro à la reconnexion. `start_ihm.sh` et le lancement des processus n'exigent plus un `.venv` précis.

J'ai restreint MongoDB à la boucle locale dans `docker-compose.yml` (il était exposé sur tout le réseau sans mot de passe), et ajouté `.env` au `.gitignore`. J'ai ajouté `GET /api/wind-forecast (IHM/app/windy.py, avec ses tests)` : elle affiche à l'opérateur la prévision de vent, mais n'envoie rien au bateau tant qu'il ne recopie pas la valeur dans `wind-command`.

7 Tests

Un test de mutation m'a montré deux tests trop faibles : en multipliant par 28 le gain de propulsion dans `AutoController.cpp`, les quatre suites restaient vertes. J'ai remplacé ces assertions par des valeurs exactes (1680 μ s pour 1 km/h sous la cible, plancher à 1600 μ s pour 0,1 km/h), fixé le bord de voile attendu quand le cap est inconnu, et ajouté des tests de bornes pour les angles (`nav_normalizeAngle`, `nav_relativeAngle`, `nav_oppositeAngle`) et pour la limite de 16 waypoints. J'ai activé `-Wall -Wextra` sur les cibles de test.

J'ai aussi remis en état la suite `gps_diag/host_test`, où un `sys.exit()` au niveau du module empêchait pytest de collecter le moindre test. État actuel : ctest 4/4, tests IHM 61 réussis, `gps_diag` 20 réussis, et les six scénarios de simulation passent. Je ne teste toujours pas `DroneApp::controlTick`, `McpwmActuators` ni `LoRaComm`, qui restent à 0 % de couverture.

8 Documentation

J'ai relu la documentation de référence du projet contre le code et corrigé les écarts : `MissionPlan` est limité à 16 waypoints et non 32, le couloir de navigation est de 30 m et non 100 m, la limitation de vitesse de variation ne s'applique qu'à l'ESC, la suite de tests n'est pas périmée (elle compile et passe), et le module `WindEstimator` n'existe pas (l'estimation est dans `AutoController`). J'ai aussi noté que `USE_OLD_NAVIGATION` ne compile plus s'il est activé, ce qui rend inutile le retour en arrière prévu, et que le commentaire de `BatteryAdc.h` parle encore de GPIO35 alors que le code lit GPIO36.

9 Ce qui reste à faire

Le premier point est de mesurer au banc la course réelle du winch et de trancher entre $\pm 20^\circ$, $\pm 90^\circ$ et $\pm 110^\circ$; rien ne doit tourner en autonomie avant. Ensuite : obtenir une clé Windy valide pour `make run-windy`, écrire des tests pour `DroneApp::controlTick`, `McpwmActuators` et `LoRaComm`, mesurer l'erreur de vent tolérable en conditions réelles, vérifier sur l'ESP32 le blocage que j'ai estimé pour l'ancienne boucle d'angle, et committer tout ce travail.

J'ai laissé de côté pour l'instant, en connaissance de cause, l'absence d'authentification de la liaison LoRa et de l'API web, qui acceptent des commandes sans identification. Sur un projet de terrain avec essais encadrés, le risque est limité ; passer les commandes en `POST` avec un secret partagé est prévu plus tard.